# Statistical Methods I: Linear Model

## Lec 06: Lack of Fit Test and Data Transformation

Zhaoxia Yu
Professor, Department of Statistics

2026-10-11

Section 1

# Intro

# Review: Lecture 01-05

- Lecture 01: LSE: $\{\hat{\beta}_0, \hat{\beta}_1\} = \arg\min_{\beta_0,\beta_1} \sum_{i=1}^{n}(y_i - \beta_0 - \beta_1 x_i)^2$

$$\hat{\beta}_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})y_i}{\sum_{i=1}^{n}(x_i - \bar{x})^2} = \sum_{i=1}^{n} \frac{(x_i - \bar{x})}{S_{XX}} y_i.$$

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x} = \sum_{i=1}^{n} \left( \frac{1}{n} - \frac{\bar{x}}{S_{XX}}(x_i - \bar{x}) \right) y_i$$

- Lecture 02: CI, and hypothesis testing related to $\beta_1$.
- Lecture 03: Estimation and prediction.
- Lecture 04: ANOVA.
- Lecture 05: Diagnostics in SLR.
- This lecture: data transformation and lack of fit test.

Section 2

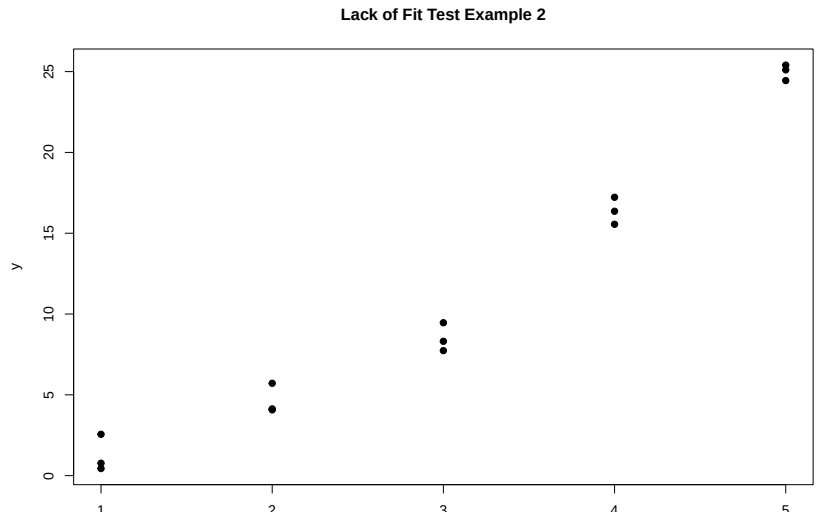
# Lack of Fit Test

# Lack of Fit Test in SLR

- We have introduced visualization methods to check the linear model assumptions.
- Goodness-of-fit tests (GOF) are often used to assess whether a statistical model/distribution fits the data well.
  - For example, the Chi-square goodness-of-fit test is used to test whether observed categorical data follow a specified distribution.
  - For simple linear regression, we consider the lack for fit test (LOF), which is an F-test, to test whether the linear regression model is a good fit for the data.

# Lack of Fit Test in SLR: Notations

- LOF test requires replicated $x$ values (i.e., multiple $y$ values for the same $x$ value).
- $k$: the number of unique $x$ values
- $n_i$: the number of replicates for the $i$th unique $x$ value
- $n = \sum_{i=1}^{k} n_i$: the total number of observations.
- $y_{ij}$: the $j$th observation of the response variable $y$ at the $i$th unique value of the predictor variable $x$, where $i = 1, 2, \ldots, k$ and $j = 1, 2, \ldots, n_i$.



# Lack of Fit Test in SLR

A LOF test compares two models:

- Reduced model $H_0$: $y_{ij} = \beta_0 + \beta_1 x_i + \epsilon_{ij}$; assumes a linear relationship between $x$ and $y$.
- Full model $H_1$: $y_{ij} = \mu_i + \epsilon_{ij}$, where $\mu_i$ is the mean of the $y$ values for the $i$th unique $x$ value; uses a separate mean for each unique $x$ value.

# Lack of Fit Test in SLR: Sum of Squares

- The reduced model ($H_0$), which is the SLR:

$$SSE_R = \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij} - \hat{y}_{ij}) = (y_{ij} - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

  where $\hat{\beta}_0, \hat{\beta}_1$ are the LSE of the SLR, and $SSE_R$ is the SEE of the SLR.

- The full model ($H_1$)):

$$SSE_F = \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij} - \bar{y}_i)^2$$

  where $\bar{y}_i$ is the mean of the $y$ values for the $i$th unique $x$ value.

- Which one is smaller? Why?

# Lack of Fit Test in SLR: F-test

- The full model has more parameters and thus fits the data at least as well as the reduced model. As a result, $SSE_F \leq SSE_R$.
- We can construct the following F-statistic to compare the SSEs:

$$F = \frac{(SSE_R - SSE_F)/(df_R - df_F)}{SSE_F/df_F}$$

  where $df_F$ and $df_R$ are the degrees of freedom for the full and reduced models, respectively.
- Under $H_0$: the linear model fits the data well, $F \sim F_{df_R - df_F, df_F}$.
- What are $df_F$ and $df_R$?

# Lack of Fit Test in SLR: Decomposition of SSE

- $SSE_R$ can be decomposed

$$
\begin{aligned}
SSE_R &= \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\hat{y}_{ij})^2 = \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\bar{y}_i+\bar{y}_i-\hat{y}_{ij})^2 \\
&= \sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\bar{y}_i)^2 + \sum_{i=1}^{k}\sum_{j=1}^{n_i}(\bar{y}_i-\hat{y}_{ij})^2 + 2\sum_{i=1}^{k}\sum_{j=1}^{n_i}(y_{ij}-\bar{y}_i)(\bar{y}_i
\\
&= SSE_F + SS_{LF} + 0 \\
&= SSE_F + SS_{LF}
\end{aligned}
$$

- where $SS_{LOF} = \sum_{i=1}^{k}\sum_{j=1}^{n_i}(\bar{y}_i-\hat{y}_{ij})^2$ is the sum of squares due to lack of fit.

# Lack of Fit Test in SLR: degrees of freedom

- We need to find the degrees of freedom for the full and reduced models.
- For the full model, we estimate $k$ means $(\mu_1, \mu_2, \ldots, \mu_k)$ from $n$ observations. Thus, $df_F = n - k$.
- For the reduced model, we estimate 2 parameters ($\beta_0$ and $\beta_1$) from $n$ observations. Thus, $df_R = n - 2$.
- Therefore, the F-statistic can be rewritten as

$$F = \frac{(SSE_R - SSE_F)/(k-2)}{SSE_F/(n-k)} = \frac{SS_{LOF}/(k-2)}{SSE_F/(n-k)}$$

- Under $H_0$: the linear model fits the data well, $F \sim F_{k-2,n-k}$.

# ANOVA Table for Lack of Fit Test in SLR

| Source | DF | SS | MS | F | P-value |
|---|---|---|---|---|---|
| Regression | 1 | $SSR$ | $MSR = \frac{SSR}{1}$ | $F = \frac{MSR}{MSE}$ | $H_0 : \beta_1 = 0$ |
| Residual | n - 2 | $SSE_R$ | $MSE = \frac{SSE_R}{n-2}$ | | |
| Lack of Fit | k - 2 | $SS_{LF}$ | $MS_{LF} = \frac{SS_{LF}}{k-2}$ | $F^* = \frac{MS_{LF}}{MSE_F}$ | $H_0 : E[y] = \beta_0 + \beta_1 x$ |
| Pure Error | n - k | $SSE_F$ | $MSE_F = \frac{SSE_F}{n-k}$ | | |

# LOF Test: Example 1

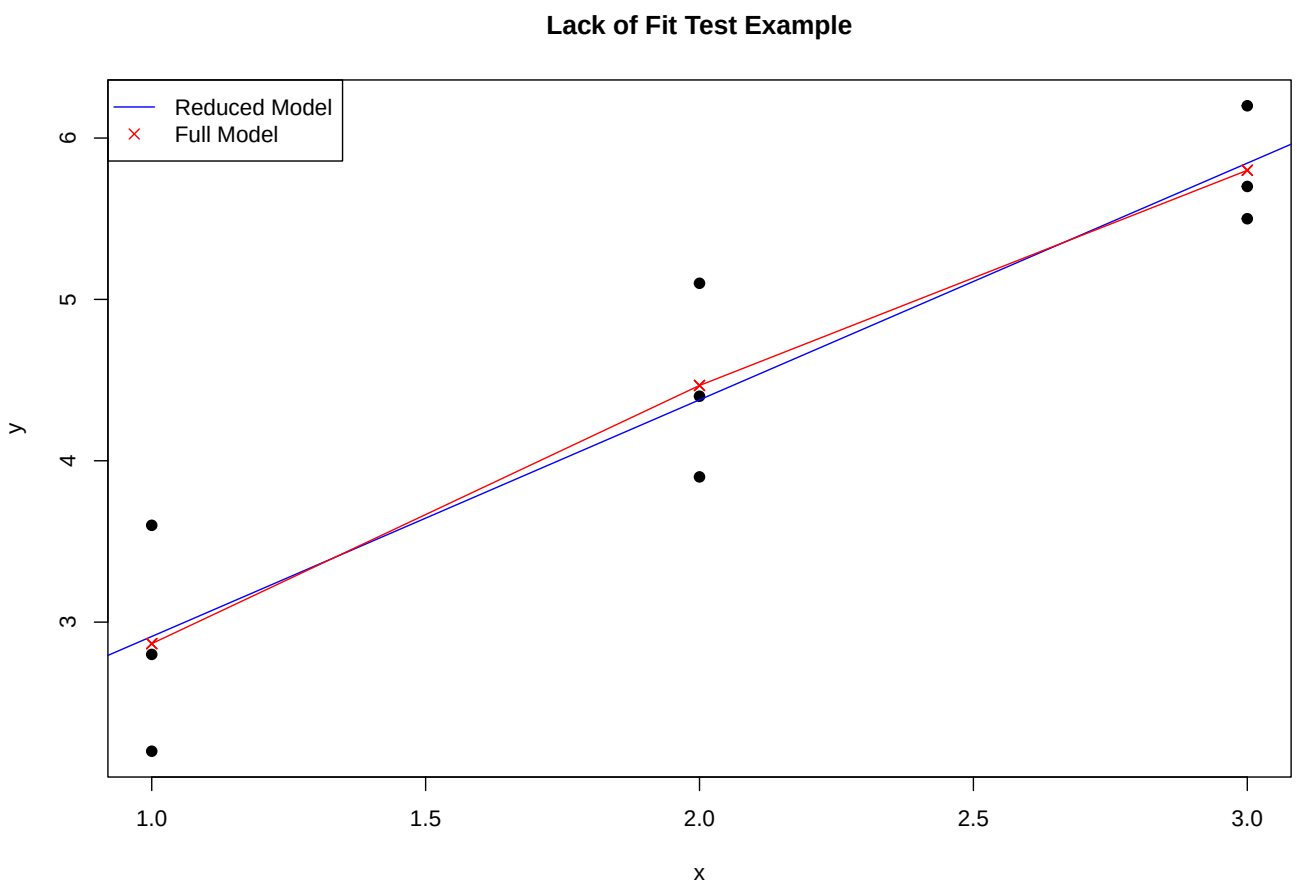
```r
# Data with replicated x values
x <- c(1, 1, 1, 2, 2, 2, 3, 3, 3)
y <- c(2.2, 2.8, 3.6, 3.9, 4.4, 5.1, 5.5, 5.7, 6.2)
data <- data.frame(x = x, y = y)
# Reduced model: y_i = beta_0 + beta_1 x_i + epsilon_i
fit_reduced <- lm(y ~ x, data = data)
# Full model: y_i = mu_i + epsilon_i
fit_full <- lm(y ~ factor(x), data = data)
anova(fit_reduced, fit_full)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ factor(x)
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1      7 2.0089
## 2      6 1.9733  1  0.035556 0.1081 0.7535
```

$$SSE_R = 2.0089, df_R = 9 - 2,\ SSE_F = 1.9733, df_F = 9 - 3.$$

# Visualize LOF: Example 1

```
plot(data$x, data$y, pch=19, xlab="x", ylab="y", main="Lack of Fit Test Example")
abline(fit_reduced, col="blue")
lines(data$x, fitted(fit_full), col="red", pch=4)
points(data$x, fitted(fit_full), col="red", pch=4)
legend("topleft", legend=c("Reduced Model", "Full Model"), col=c("blue", "red"), pch=c(NA, 4), lty=c(1, NA
```



# Interpretation of LOF: Example 1

- The ANOVA table shows the comparison between the reduced and full models.
- The plot shows the data points, the fitted line from the reduced model (in blue), and the fitted means from the full model (in red). They are very close to each other.
- The p-value for the LOF test is 0.75, which is greater than the significance level of 0.05.
- Therefore, we do not reject the null hypothesis and conclude that the linear model fits the data well.

# LOF: Example 2

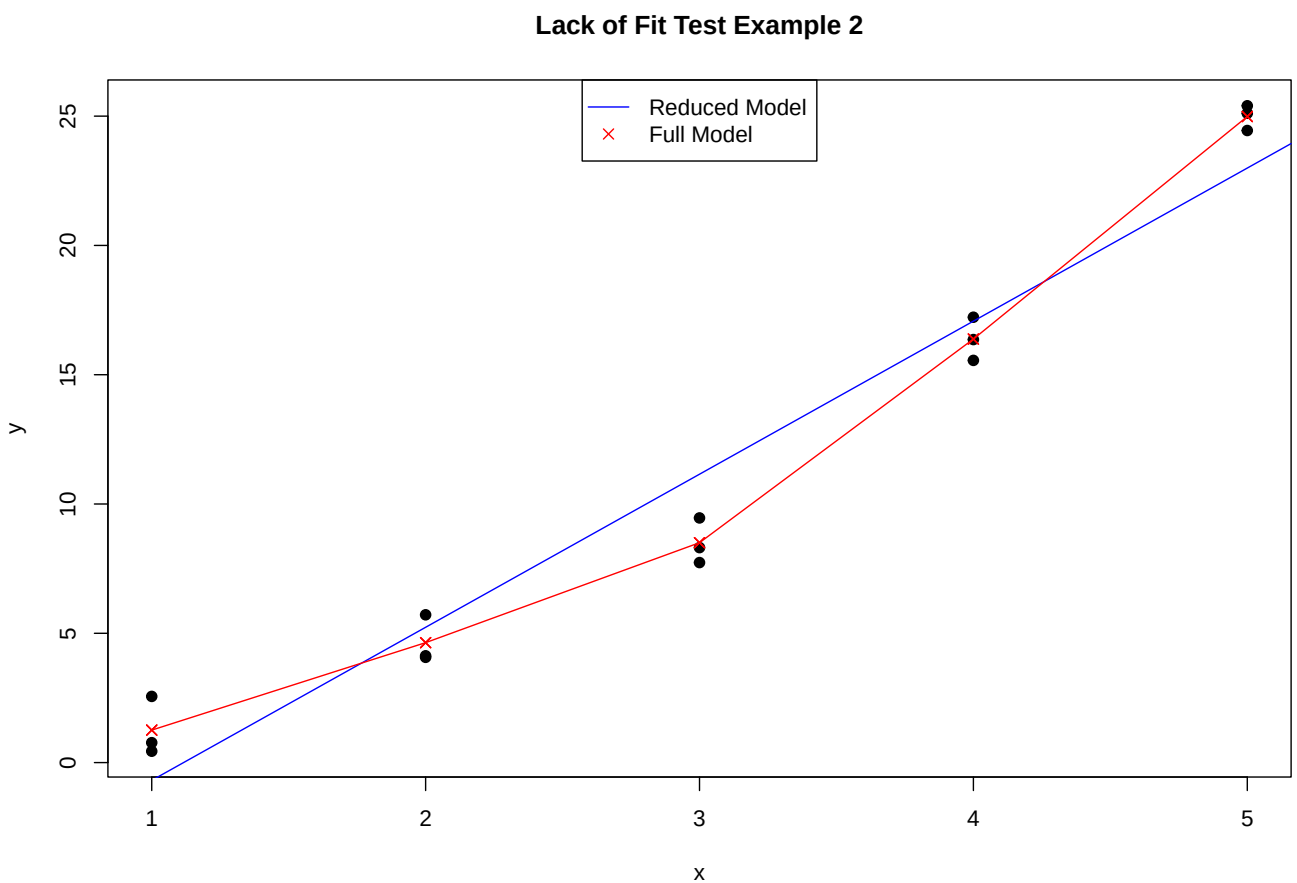
- In this example, the true model is not linear

```
set.seed(123)
x <- rep(1:5, each=3)
y <- x^2 + rnorm(15, 0, 1) # true model is quadratic
data <- data.frame(x = x, y = y)
# Reduced model: y_i = beta_0 + beta_1 x_i + epsilon_i
fit_reduced <- lm(y ~ x, data = data)
# Full model: y_i = mu_i + epsilon_i
fit_full <- lm(y ~ factor(x), data = data)
anova(fit_reduced, fit_full)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ factor(x)
##   Res.Df    RSS Df Sum of Sq      F    Pr(>F)
## 1     13 54.565
## 2     10  7.760  3    46.804 20.104 0.0001479 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

# Visualize LOF: Example 2

```
plot(data$x, data$y, pch=19, xlab="x", ylab="y", main="Lack of Fit Test Example 2")
abline(fit_reduced, col="blue")
lines(data$x, fitted(fit_full), col="red", pch=4)
points(data$x, fitted(fit_full), col="red", pch=4)
legend("top", legend=c("Reduced Model", "Full Model"), col=c("blue", "red"), pch=c(NA, 4), lty=c(1, NA))
```



# Interpretation of LOF: Example 2

- The ANOVA table shows the comparison between the reduced and full models.
- The plot shows the data points, the fitted line from the reduced model (in blue), and the fitted means from the full model (in red). They are quite different.
- The p-value for the LOF test is 0.0001, which is less than the significance level of 0.05.
- Therefore, we reject the null hypothesis and conclude that the linear model does not fit the data well.

# LOF Test: When replicated x values are not available

- When there is no replicated $x$ values, we cannot perform the LOF test.
- There are several options. For example, Linear vs polynomial (quadratic/cubic)

```
set.seed(12)
x=seq(1, 10, by=0.5)
y=(x-3)^2 + rnorm(length(x), 0, 2)
d=data.frame(x, y)
fit_lin  <- lm(y ~ x, data = d)
fit_poly2 <- lm(y ~ x + I(x^2), data = d)
fit_poly3 <- lm(y ~ x + I(x^2) + I(x^3), data = d)
anova(fit_lin, fit_poly2)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ x + I(x^2)
##   Res.Df     RSS Df Sum of Sq      F    Pr(>F)
## 1     17 1008.50
## 2     16   46.65  1    961.85 329.92 4.202e-12 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
anova(fit_lin, fit_poly3)
```

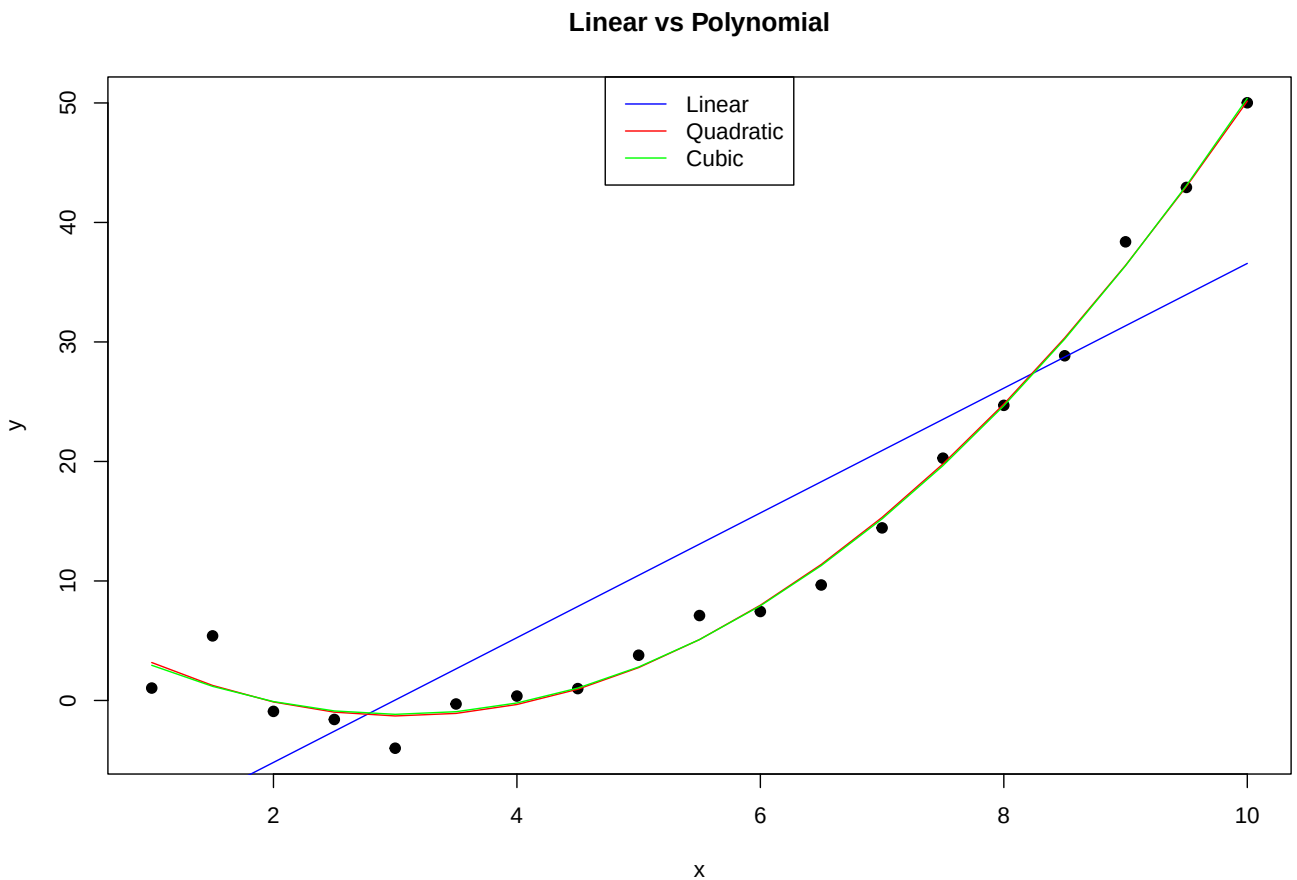
```
## Analysis of Variance Table
##
## Model 1: y ~ x
## Model 2: y ~ x + I(x^2) + I(x^3)
##   Res.Df     RSS Df Sum of Sq      F    Pr(>F)
## 1     17 1008.50
## 2     15   46.37  2    962.13 155.62 9.315e-11 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
anova(fit_poly2, fit_poly3)
```

```
## Analysis of Variance Table
##
## Model 1: y ~ x + I(x^2)
## Model 2: y ~ x + I(x^2) + I(x^3)
##   Res.Df    RSS Df Sum of Sq      F Pr(>F)
## 1     16 46.647
## 2     15 46.370  1   0.27721 0.0897 0.7687
```

# Visualize Linear vs Polynomial

```r
plot(d$x, d$y, pch=19, xlab="x", ylab="y", main="Linear vs Polynomial")
lines(d$x, fitted(fit_lin), col="blue")
lines(d$x, fitted(fit_poly2), col="red")
lines(d$x, fitted(fit_poly3), col="green")
legend("top", legend=c("Linear", "Quadratic", "Cubic"), col=c("blue", "red", "green"), lty=1)
```



# When Assume Model Is not Reasonable/Adequate

- When one or several assumptions of the linear model are violated, what can we do?
- Consider more complex models. For example,
  - If the relationship between $x$ and $y$ is not linear, we may consider polynomial regression or other types of regression (e.g., spline regression, generalized additive models).
  - If the variance of the errors is not constant, we may consider weighted least squares or generalized least squares.
- We may consider data transformation.

Next, we present how to transform the data to address non-linearity or non-constant variance.

Section 3

# Data Transformation

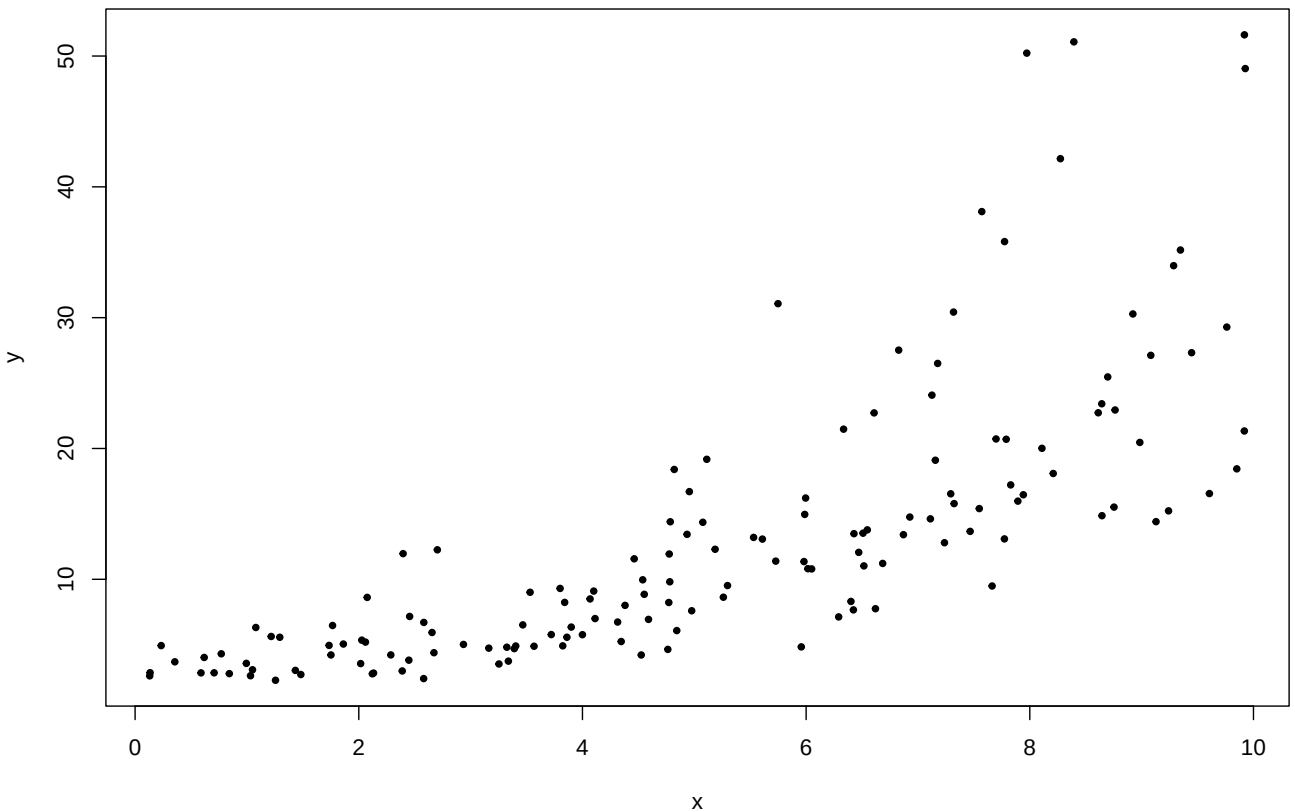
# Data Transformation: General Rules

- Transformations are often used to achieve linearity, constant variance, and normality of errors.
- Transform $x$ or $y$?
- Not linear: If the relationship between $x$ and $y$ is not linear but residuals are generally normal and have constant variance, we may consider transforming $x$.
- Not constant variance: If the variance of $y$ varies with $x$, consider transforming $y$ (e.g., log or square root).
- If both the relationship and the variance are problematic, we may consider transforming $y$ to stabilize variance first and then try transforming $x$.

# Data Transformation: General Rules

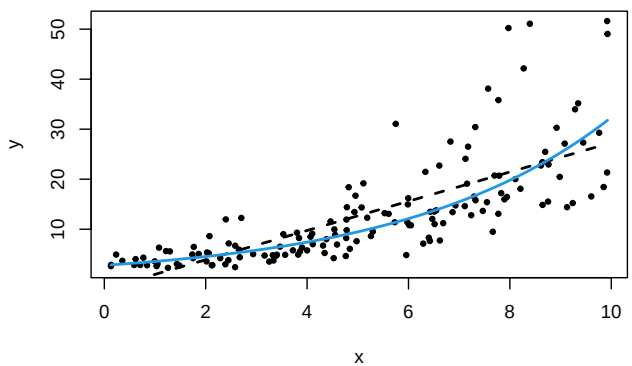
- Common transformations include:
  - Logarithmic transformation: $y' = \log(y)$ or $x' = \log(x)$
  - Square root transformation: $y' = \sqrt{y}$ or $x' = \sqrt{x}$
  - Reciprocal transformation: $y' = 1/y$ or $x' = 1/x$
  - Box-Cox transformation: a family of power transformations that includes the above as special cases.
- The choice of transformation depends on the specific characteristics of the data and the goals of the analysis.

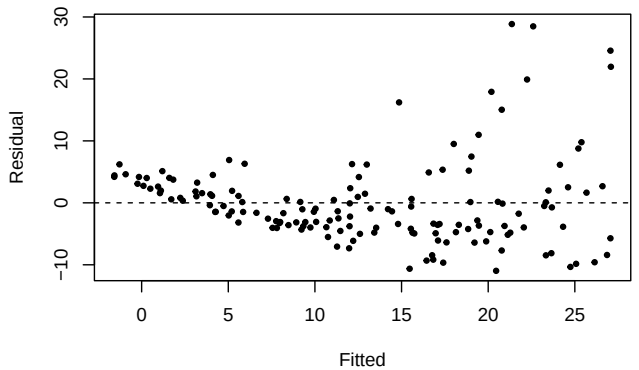
# Example 1

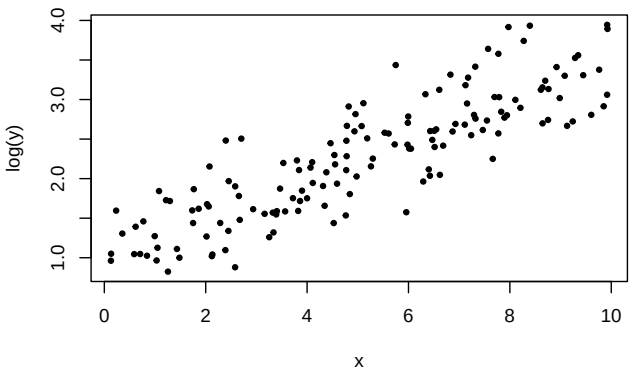


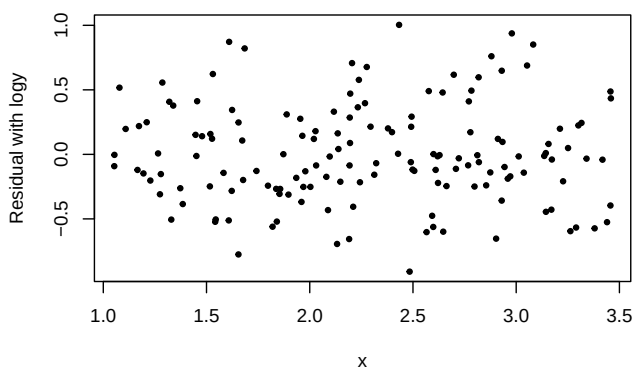

- Transform $x$ or $y$? Why?

# Example 1: : Original vs Transformed



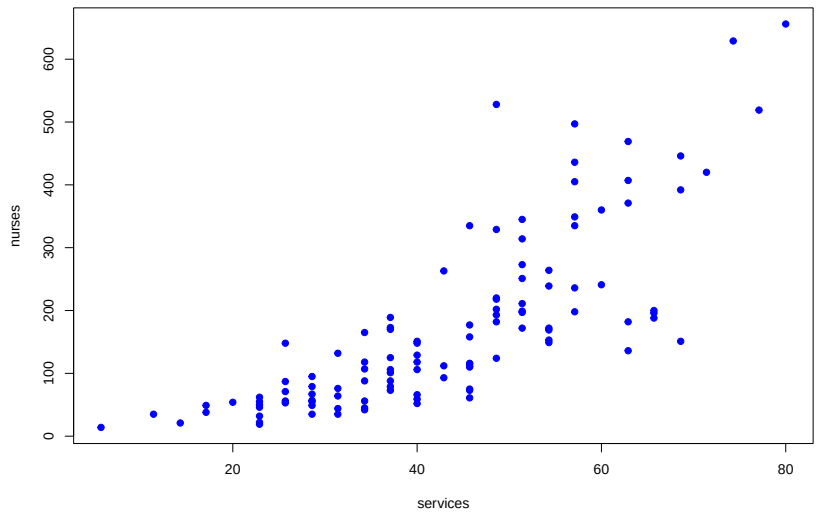



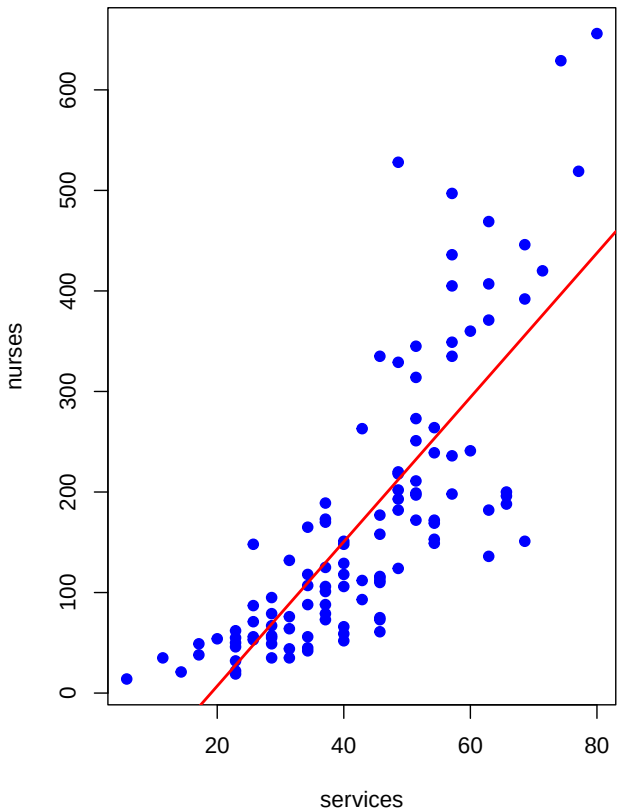

- Transform $x$ or $y$? Why?

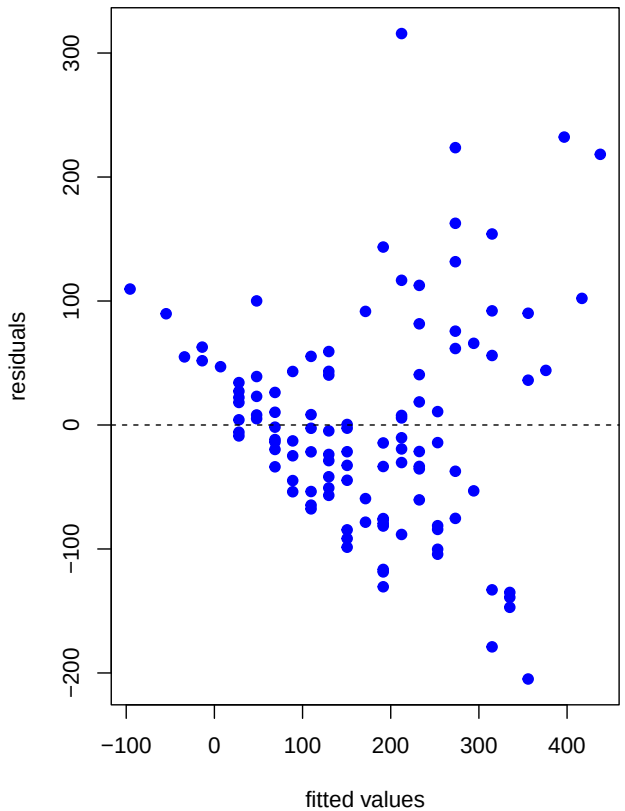
# Example 2: SENIC




- A random sample of 113 hospitals taken from a Study in the Efficacy of Nosocomial Infection Control (SENIC Project).
- nurses: Average number of full-time equivalent registered and licensed practical nurses
- service: Percent of 35 potential facilities and services provided by the hospital

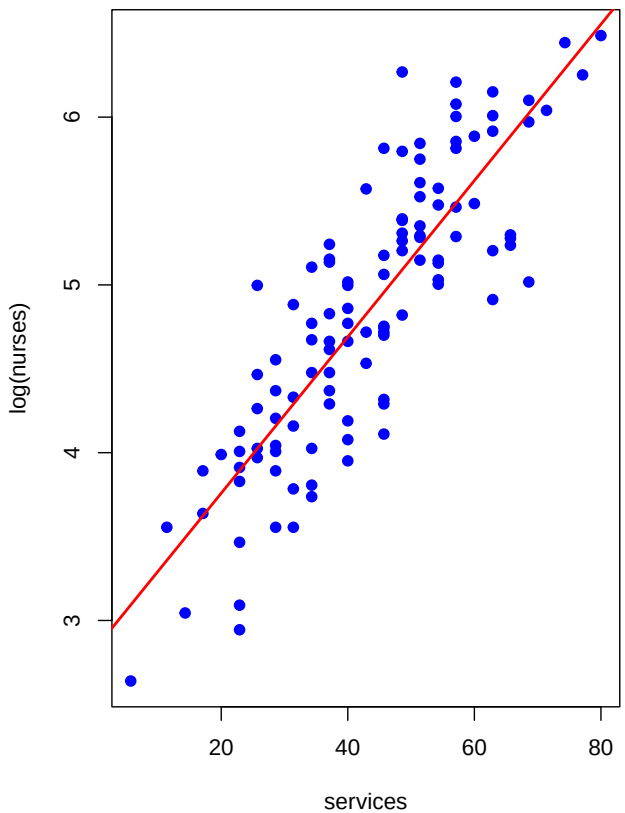
# Example 2: LM Fit and Residuals

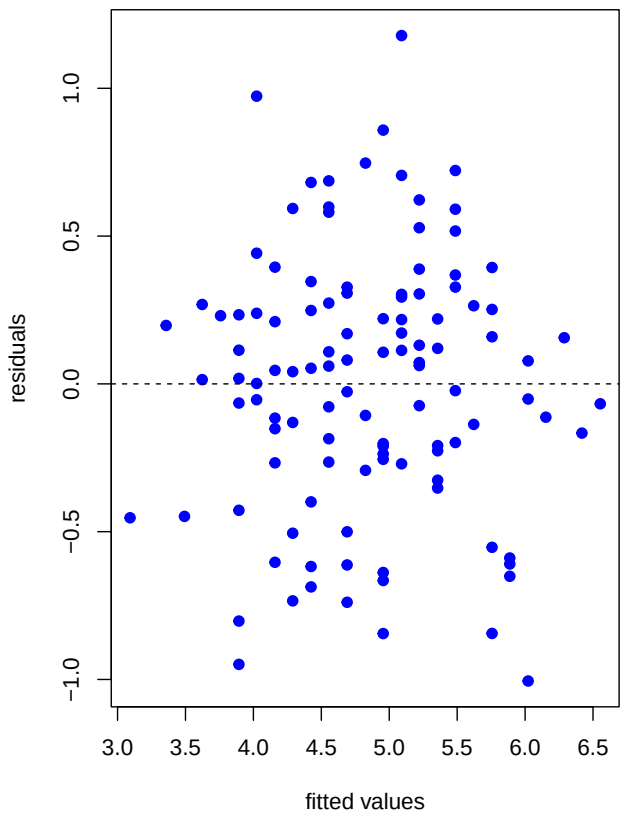


# Example 2: Transform y

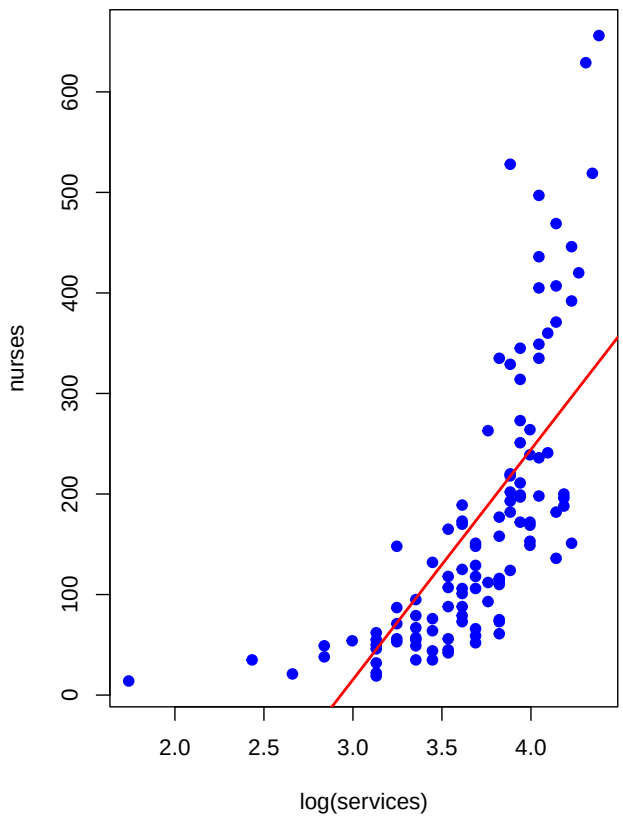
- Let's try log transformation on y.

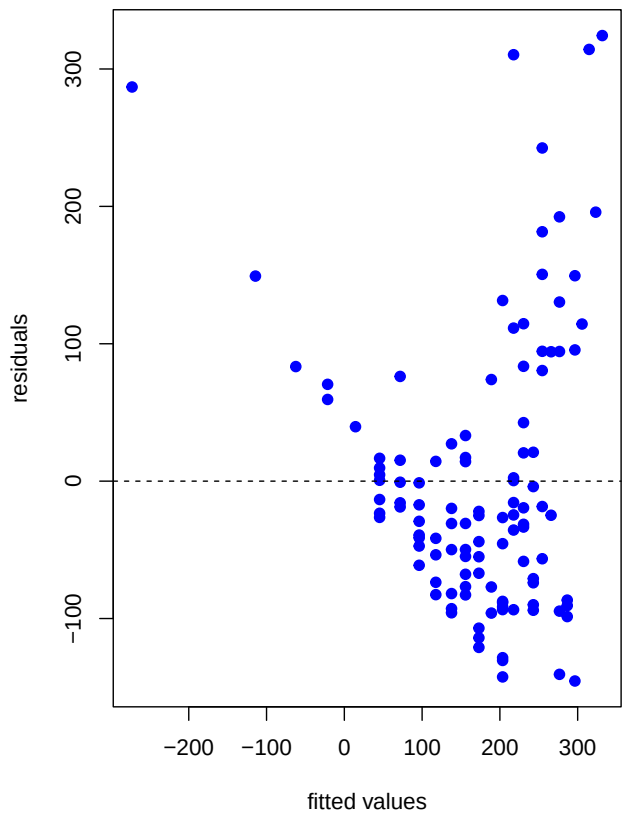


# Example 2: Transform x

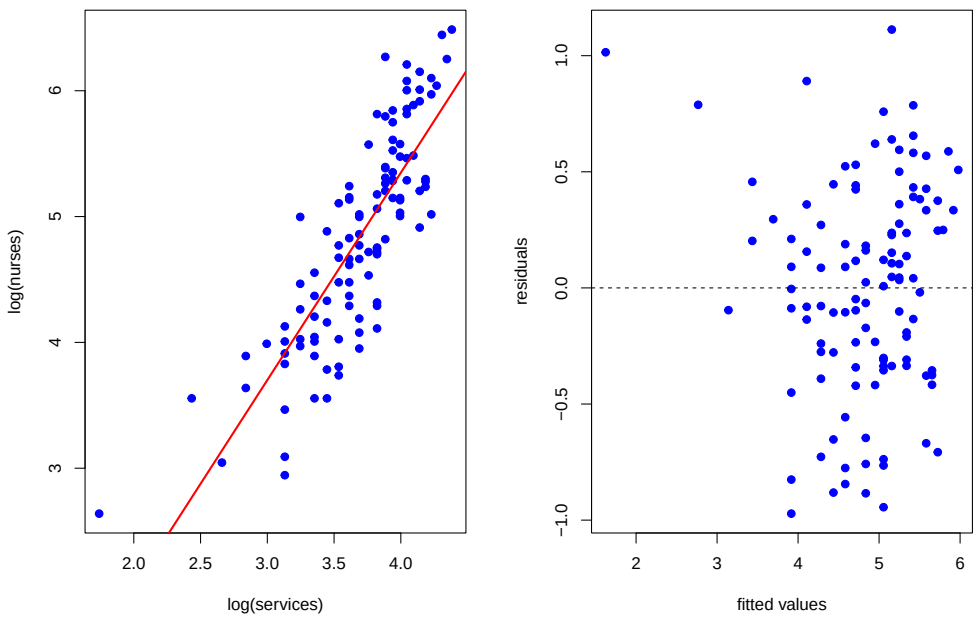
- Let's try log transformation on x.



# Example 2: Transform both x and y

- Let's try log transformation on both x and y.




- What is your choice?

# Example 2: Results and Interpretation

```
summary(senic_logy_fit)
```

```
##
## Call:
## lm(formula = log(nurses) ~ services, data = senic)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.00519 -0.26450  0.04573  0.26862  1.17838
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept) 2.826546   0.125500   22.52   <2e-16 ***
## services    0.046588   0.002744   16.98   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.4414 on 111 degrees of freedom
## Multiple R-squared:  0.722,  Adjusted R-squared:  0.7195
## F-statistic: 288.2 on 1 and 111 DF,  p-value: < 2.2e-16
```

# Example 2: Results and Interpretation

- Fitted model:

$$\log(\text{Nurses}) = \hat{\beta}_0 + \hat{\beta}_1 \, \text{Services}, \qquad \hat{\beta}_1 = 0.047.$$

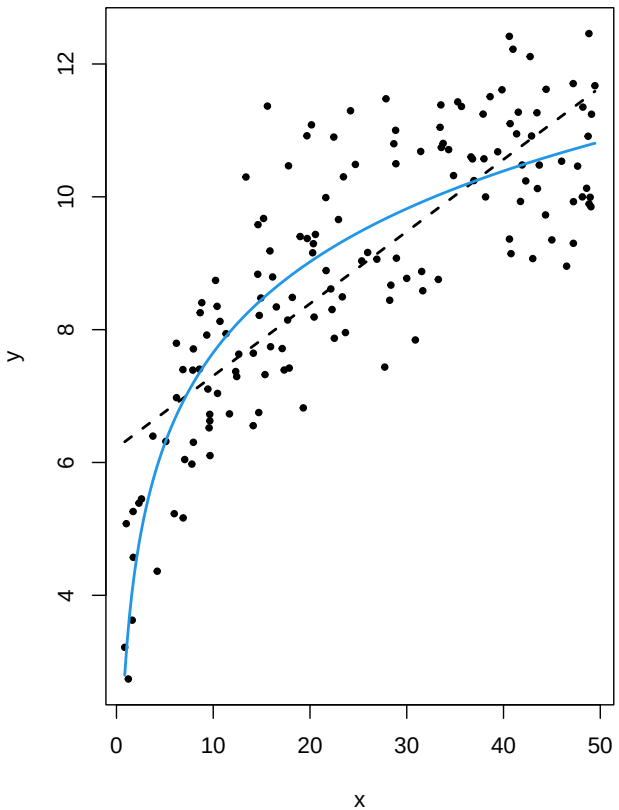
For a 1-unit increase in Services (from $x$ to $x+1$),

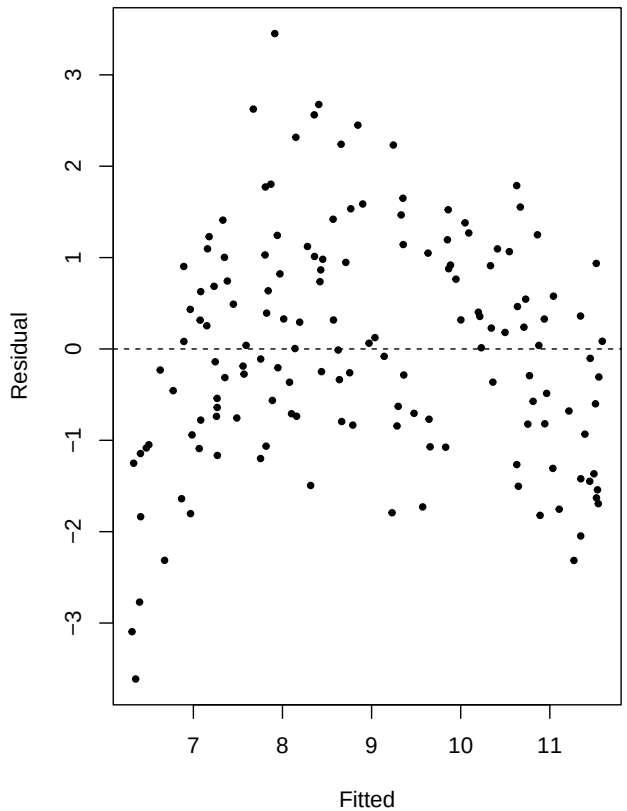
$$\log(\text{Nurses} \mid x+1) - \log(\text{Nurses} \mid x) = \hat{\beta}_1,$$

$$\Rightarrow \quad \frac{\text{Nurses} \mid x+1}{\text{Nurses} \mid x} = \exp(\hat{\beta}_1).$$

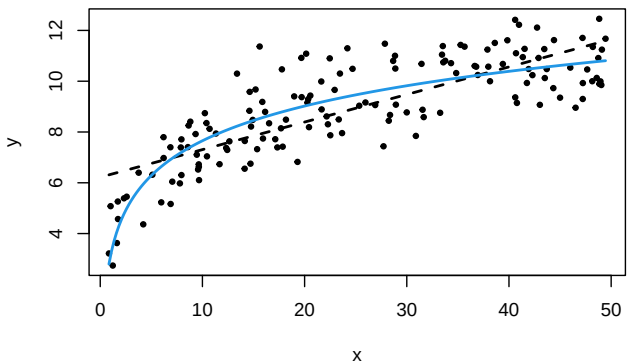
- Thus the expected (and median) number of nurses is - *multiplied* by $\exp(\hat{\beta}_1)$ for each $+1$ in Services. With $\hat{\beta}_1 = 0.047$: $\exp(0.047) \approx 1.048$, i.e., about a **4.8% increase per unit**.
- How about for k-unit increase in services?

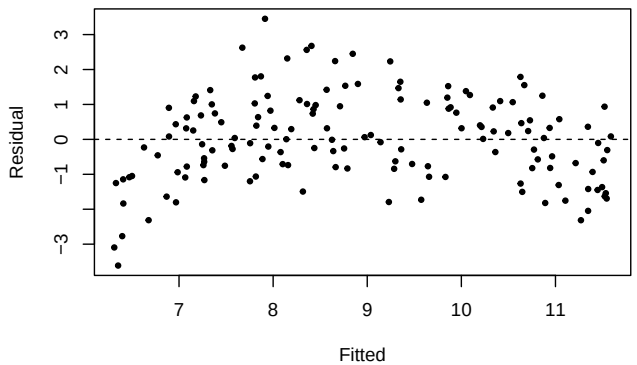
# Example 3:

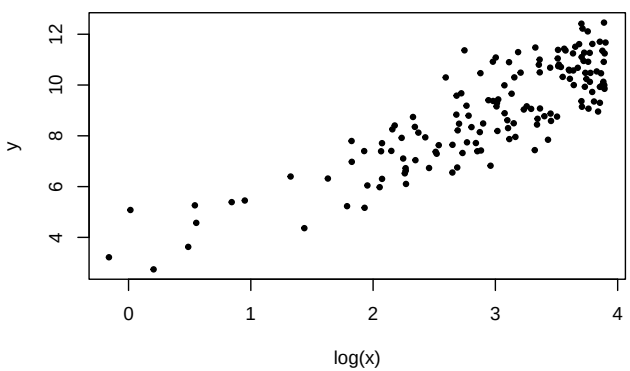


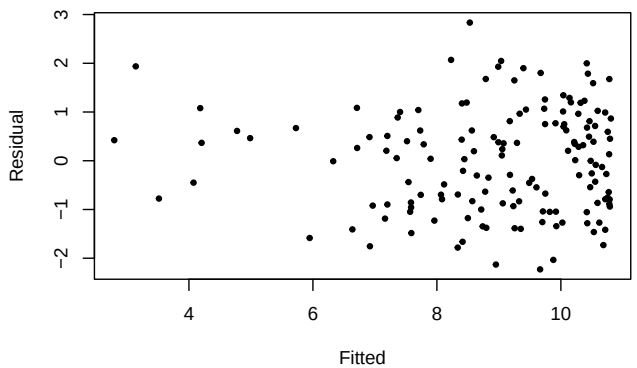



# Example 3: Original vs Transformed

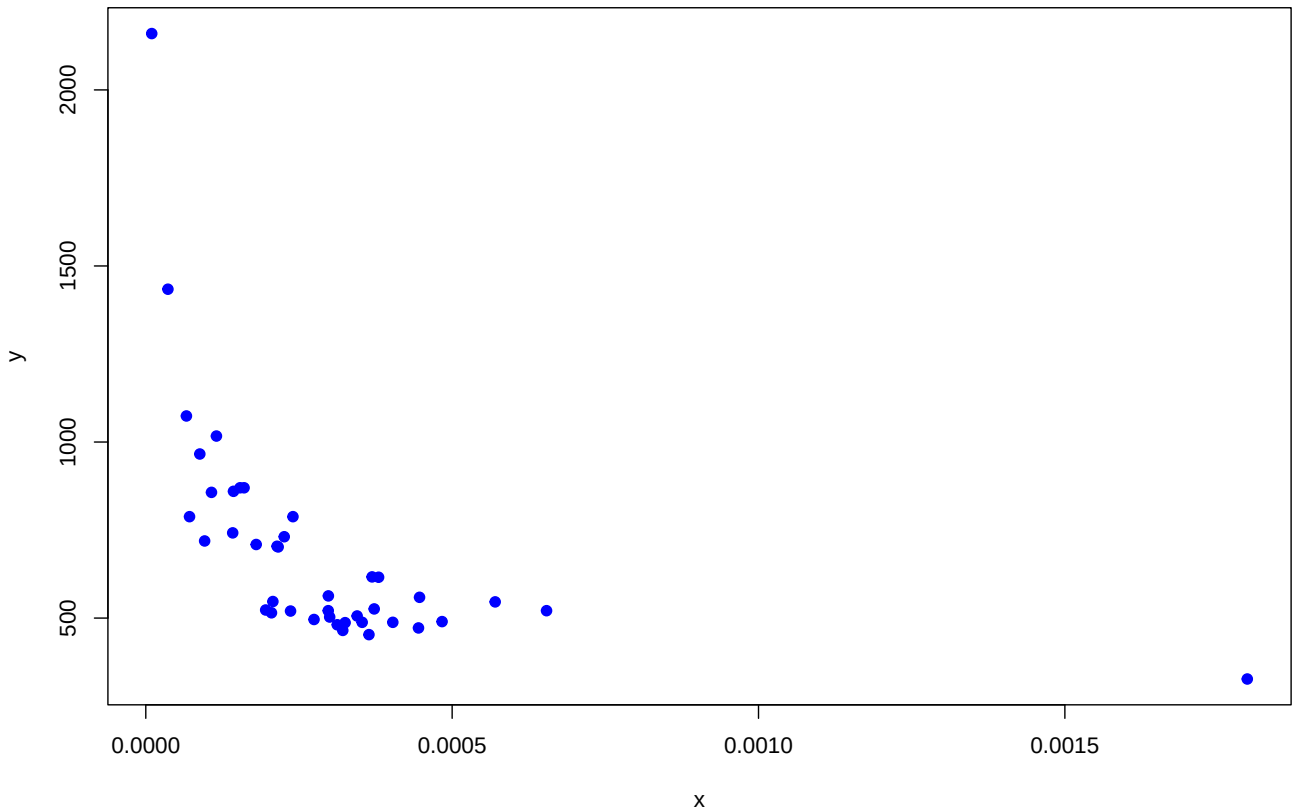


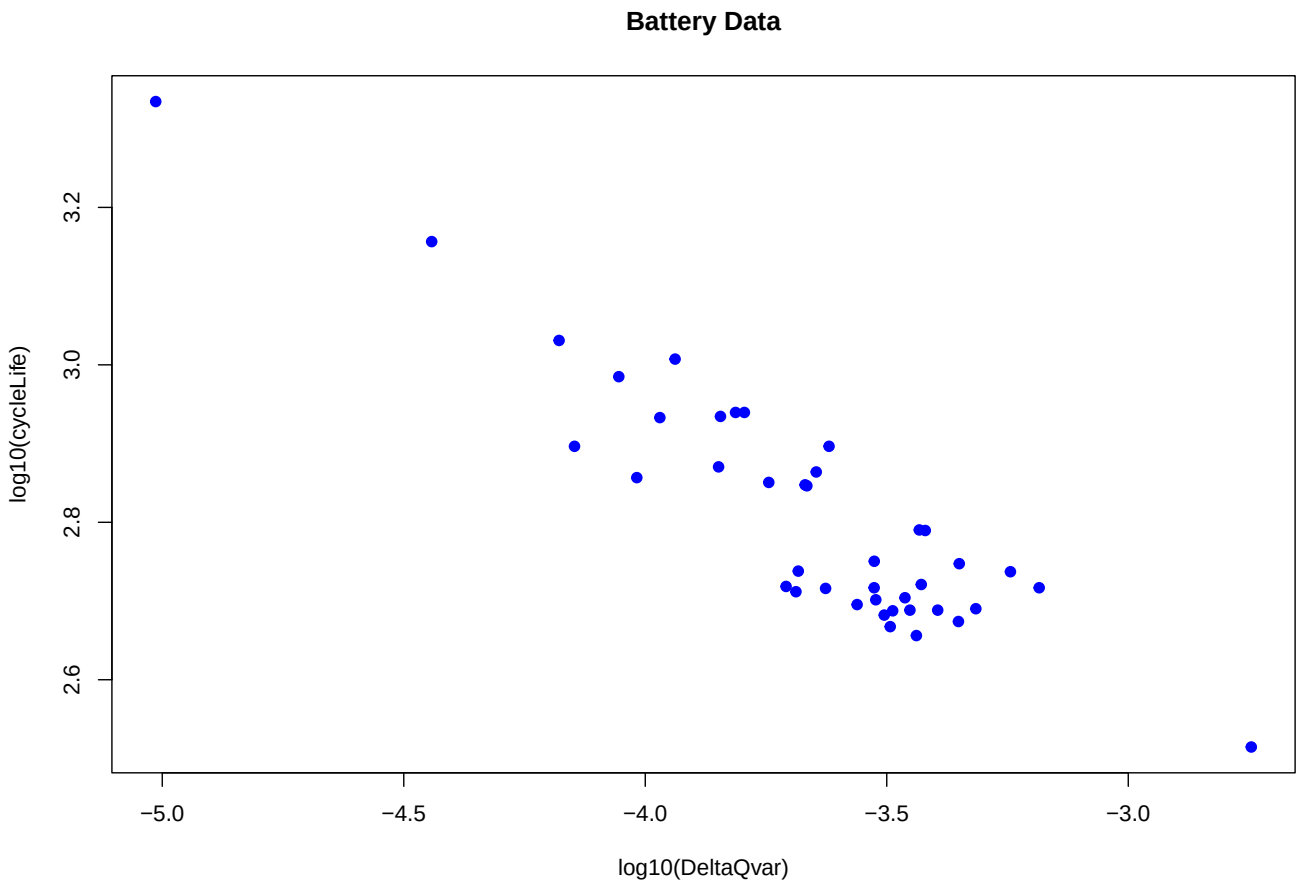



# Example 4



# Example 4: Battery Data



# Example 4: Results

```
fit <- lm(log10(cycleLife) ~ log10(DeltaQvar), data = battery)
summary(fit)
```

```
##
## Call:
## lm(formula = log10(cycleLife) ~ log10(DeltaQvar), data = battery)
##
## Residuals:
##       Min        1Q    Median        3Q       Max
## -0.107697 -0.056378  0.009384  0.056817  0.103507
##
## Coefficients:
##                  Estimate Std. Error t value Pr(>|t|)
## (Intercept)       1.44364    0.09990   14.45   <2e-16 ***
## log10(DeltaQvar) -0.37281    0.02718  -13.72   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.06424 on 39 degrees of freedom
## Multiple R-squared:  0.8283, Adjusted R-squared:  0.8239
## F-statistic: 188.2 on 1 and 39 DF,  p-value: < 2.2e-16
```

- Interpretation of the slope: A 1% increase in DeltaQvar is associated with an approximate ____ decrease in cycleLife, on average.

Section 4

# Box-Cox Transformation

# Box-Cox Transformation

- Box-Cox transformation is defined as:

$$y^{(\lambda)} = \begin{cases} \frac{y^{\lambda}-1}{\lambda}, & \lambda \neq 0 \\ \log(y), & \lambda = 0 \end{cases}$$

  E.g., when $\lambda = 0.5$, it is a square root transformation, i.e., $y' = \sqrt{y}$.

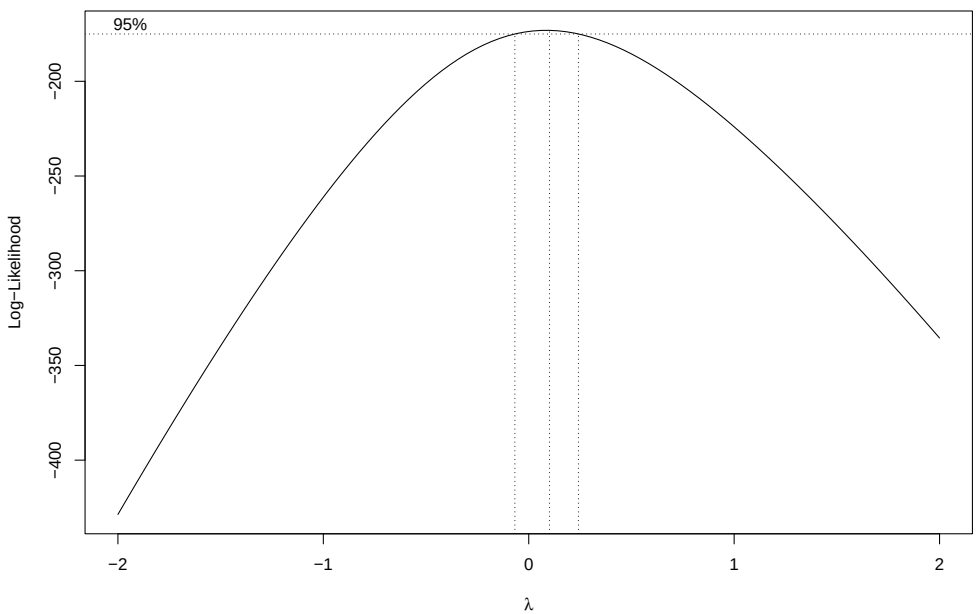
- How to find $\lambda$? Box-Cox transformation finds the optimal $\lambda$ by maximizing the log-likelihood function.

$$y_i^{(\lambda)} = \beta_0 + \beta_1 x_i + \epsilon_i, \quad \epsilon_i \overset{iid}{\sim} N(0, \sigma^2)$$

- The fucntion "boxcox'' in MASS package in R can be used to find the optimal $\lambda$.

# Box-Cox Transformation for SENIC Data

- In this example, the optimal value of $\lambda$ is around 0, which suggests a log transformation on y.



# Box-Cox Transformation for SENIC Data

```
library(MASS)
senic_fit <- lm(nurses ~ services, data = senic)
boxcox(senic_fit, lambda = seq(-2, 2, by = 0.1),
       xlab = expression(lambda),
       ylab = "Log-Likelihood",
       main = "Box-Cox Transformation for SENIC Data")
```

Section 5

# Conclusion

# Conclusion



*"essentially, all models are wrong, but some are useful"*

# Conclusion



*"essentially, all models are wrong, but some are useful"*
— George Box (October 1919 - March 2013)